Predicting Hit Songs From Audio Features Using Ensemble and Deep Learning Models

By: Steven Li (listeven@g.ucla.edu), Daniel Yao (daniel50@ucla.edu), Tim Wu (twu888@g.ucla.edu), and Ethan Biddle (ethanbiddle31@g.ucla.edu)
University of California, Los Angeles
Applied Statistics and Data Science Master's Program
March 19th, 2026

Abstract

This project explores whether a song's success can be predicted using only its audio features in order to help record labels and artists. We frame the problem as a binary classification task to distinguish hit songs from non-hit songs using a dataset of Billboard Hot 100 tracks (1958-2021) combined with Spotify audio features. A song is labeled as a hit based on chart performance, specifically if it remains on the chart at least 15 weeks and achieves a peak position of 10 or above. We begin with baseline models, including logistic regression and gradient boosted trees (**XGBoost**, **LightGBM**, and **CatBoost**). For our advanced models, we used a **Wide & Deep neural network** and **TabNet**. To further improve performance, we incorporated a stacked and weighted ensemble that combined the predictions from our gradient boosted trees. Model performance is evaluated using AUPRC and ROC-AUC to account for class imbalance. The ensemble achieves the strongest performance, outperforming individual models and exceeding random baseline results, though improvements plateau across increasingly complex models. These results suggest that while audio features provide some predictive signal, they are insufficient to fully explain why songs become hits. This also highlights the importance of external factors besides music features such as promotions, artist popularity, and social influence.

Introduction & Motivation

Millions of tracks are released annually across hundreds of digital platforms, making the identification of songs with high commercial success increasingly important for artists and record labels. Predictive modeling of song success has implications for resource allocation, marketing strategy, and recommendation system optimization. Advancements in music information retrieval and large-scale audio analysis have enabled structured extraction of intrinsic musical features from recorded tracks. These features provide a consistent and measurable representation of musical content that can be used to investigate predictive relationships between audio features and listener engagement outcomes.

Musical perception arises from complex interactions among attributes like loudness, timbre, rhythm, valence, and energy. Oftentimes, these interactions are non-linear and context-dependent, presenting challenges for traditional statistical modeling approaches. Machine learning methods, on the other hand, offer flexible modeling frameworks capable of capturing high-dimensional dependencies amongst features. Deep learning architectures, in particular, enable hierarchical representation learning that can potentially capture perceptual patterns associated with listener preferences and commercial performance.

This study investigates the extent to which intrinsic musical features extracted via the Spotify API contain predictive signals related to a song's success. The primary objective is to quantify the extent to which audio-derived representations alone can support reliable discrimination between hit and non-hit songs. By focusing exclusively on audio-derived attributes, the analysis provides a controlled empirical framework for evaluating the predictive capacity of musical structure independent of broader industry or consumption dynamics. Furthermore, understanding the predictive limits of intrinsic musical features is important for both methodological development in music analytics and practical applications in content discovery and recommendation systems.

We formulated the song hit prediction problem as a supervised binary classification, where the objective is to determine whether a track exceeds a predefined threshold. The threshold combines peak chart position and chart longevity to capture both the intensity and duration of a song's popularity. The model's performance is then evaluated using metrics that reflect predictive discrimination under class imbalance. We used **The Area Under the Precision-Recall Curve** (AUPRC) as our primary evaluation metric due to the inherent class imbalance in hit song prediction, where successful tracks constitute a minority of the dataset (~14%). And unlike **accuracy** or **AUC-ROC**, AUPRC provides a more informative assessment of model performance by emphasizing the precision-recall trade-off for the positive class. This is particularly important in practical settings, where correctly identifying true hits while limiting false positive predictions is critical for decision-making in music promotion and investment.

Ultimately, by systematically evaluating deep learning models within a controlled feature-based framework, this study aims to provide a clearer understanding of both the predictive potential and limitations of intrinsic musical features in modeling commercial success.

Related Works

Predicting song popularity has been an active area of research. Early work applied SVMs and boosting classifiers to a corpus of 1700 songs using acoustic and lyric features, concluding that lyric-based signals outperformed audio alone in distinguishing hits from non-hits. This finding is a recurring problem in related works and our own experiment (Dhanaraj & Logan, 2005). Audio features like tempo, time signature, loudness, and genre provide some signal, but predictive accuracy across classification techniques remains consistently low. This is further agreed with subsequent work, where they argued that "hit song science" remains fundamentally limited. The results showed that some subjective labels may be learned from audio features, but not their popularity (Pachet & Roy, 2008).

More recent work has revisited the problem using deep learning, experimenting with CNNs trained on raw mel-spectrograms, jointly learning audio features and prediction models rather than relying on hand-crafted features (Yang et al. 2017). Their approach differs from previous studies by using end-to-end learned representations directly from audio waveforms. While deep structures outperformed shallow ones, gains were modest. This suggests that the bottleneck lies in the available signal rather than the model architecture. A related deep learning approach, HSP-TL (Vavaroutsos et al., 2024), combines temporal information with audio and lyric features using a triplet loss function. They found that incorporating lyrics improves song uniqueness and better reflects music trends.

Similarly, other research demonstrated that combining audio with additional modalities, such as visual features from music videos, have shown accuracy improvements of around 9% over audio-only baselines (Chin et al. 2025). This reinforces the idea that audio features alone have a natural ceiling. The related works have a shared conclusion that aligns with our own findings: audio characteristics are predictive but insufficient. External factors, such as promotions, artist reputation, appear to drive much of what makes a song hit. No purely audio-based model has yet to overcome this problem.

Dataset Overview

The dataset used in this study was obtained from a publicly available Kaggle repository containing detailed audio feature information alongside historical chart performance data from the **Billboard Hot 100**. Specifically, the file leaderboard.csv includes all songs that appeared on the weekly Billboard Hot 100 chart from August 1, 1959 to May 28, 2021. This file provides information such as chart position, artist name, previous week's position, peak position, and total weeks on the chart.

For the purposes of this study, the variables peak position and weeks on chart were used to define a binary label indicating whether a song was considered a "hit." Songs that spent at least 15 weeks on the chart and achieved a peak position within the top 10 were classified as hits. Based on this threshold, approximately 14.3% of songs in the dataset were labeled as hits, resulting in a notable class imbalance.

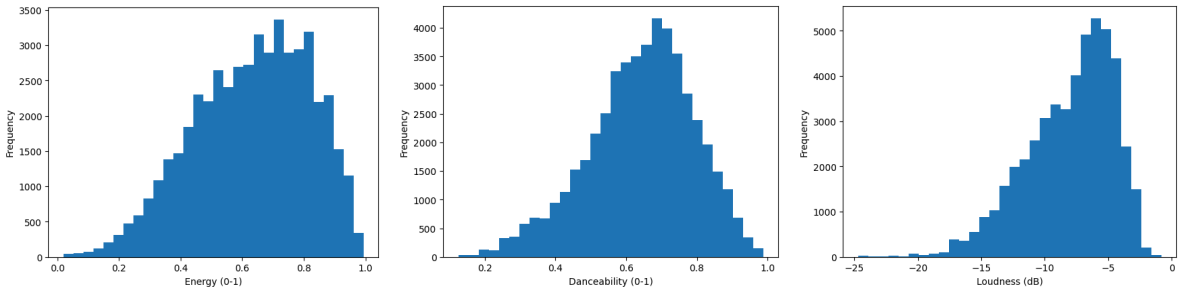
A second file contained intrinsic audio features extracted from the **Spotify API**, including track duration, explicit content indicator, energy, musical key, loudness, valence, tempo, and additional acoustic descriptors. Initially, this file contained approximately 29,502 songs. To avoid extensive imputation procedures, all observations with missing values were removed, resulting in a final dataset of approximately 24,719 songs.

Additional feature engineering steps were performed to capture interactions between musical characteristics. For example, the product of tempo and energy was used to represent overall musical intensity or perceived "upbeatness," while energy multiplied by danceability was intended to capture rhythmic groove. Similarly, the interaction between valence and energy was introduced to model more complex relationships between emotional tone and musical dynamism.

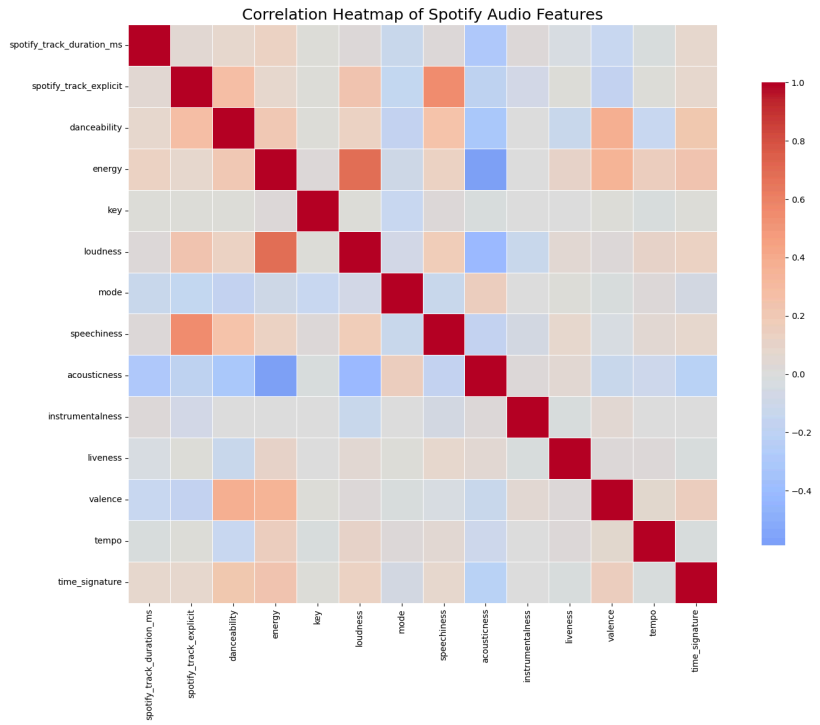
Genre information was also restructured. The original dataset contained approximately 1,045 unique genre labels, expressed as concatenated strings (e.g., "pop rock," "rock," "pop"). To address sparsity and semantic redundancy, genre information was transformed into a learned embedding representation with 16 dimensions. This approach enabled a more structured representation of stylistic similarity between tracks while reducing the dimensionality of categorical inputs**.**

genre_emb_1	genre_emb_2	genre_emb_3	genre_emb_4	genre_emb_5	genre_emb_6	genre_emb_7	genre_emb_8	genre_emb_9	genre_emb_10
0.171244	-0.306219	0.030578	0.666900	-0.424515	0.171154	0.131719	0.045151	0.427118	-0.121575
-0.179796	0.368559	-0.099204	0.570811	-0.116604	0.604052	-0.022766	0.445687	-0.500438	-0.429720
-1.217189	-0.106377	-1.078668	-0.949431	1.001791	0.462621	-0.307503	-0.354504	-0.219591	-0.472916
-1.267263	0.424386	-0.796611	-1.003890	0.538342	0.984852	-0.148557	-0.604630	0.035902	-0.210625
-0.128825	-0.155267	0.260334	0.718790	-0.273618	0.058279	0.528503	0.145543	-0.255186	-0.392838

An extensive exploratory analysis was conducted to examine feature distributions and relationships. A key observation was the substantial variation in feature scales. For instance, loudness is measured in decibels and typically ranges from approximately −30 to 0, whereas features such as energy, valence, and danceability are defined on normalized scales between 0 and 1. Without appropriate preprocessing, features with larger numerical magnitudes may dominate gradient updates or distance-based computations, potentially biasing model training.



Correlation analysis among numeric features revealed several notable patterns. Strong positive correlations were observed between loudness and energy, reflecting the intuitive relationship between perceived intensity and amplitude. Additionally, the explicit content indicator showed associations with features characteristic of spoken-word or lyric-driven musical styles. Conversely, weak correlations were observed between acousticness and features such as loudness and energy, suggesting that acoustic tracks may exhibit distinct structural characteristics not directly tied to amplitude-based descriptors.



Following the feature engineering procedures described in Section 4, numerical variables were standardized prior to model training to address disparities in feature scale. Standardization was implemented using z-score normalization via the StandardScaler. The scaling parameters (mean and standard deviation) were estimated exclusively on the training set and subsequently applied to the validation and test sets. This procedure prevents information leakage and ensures that model evaluation reflects generalization to unseen data.

Furthermore, to account for class imbalance, stratified sampling was used when constructing the training, validation, and test subsets. This approach preserves class proportions across splits and provides more stable performance estimates compared to purely random partitioning.

The dataset was partitioned into training, validation, and test sets using a 70-15-15 split. The validation set was used for model selection and hyperparameter tuning, while the test set remained fully held-out until final performance evaluation. This separation reduces the risk of overfitting during model development and supports an unbiased assessment of model generalization.

Methodology

We frame hit song prediction as a binary classification problem. Given a feature vector x representing the audio attributes of a song. The goal is to learn a function f(x) -> {0,1}, where 1 denotes a hit and 0 denotes a non-hit. A song is labeled a hit if it achieves a peak chart position of 10 or lower and remains on

the Billboard Hot 100 for at least 15 weeks. All models are trained on the same feature set and evaluated using AUPRC and ROC-AUC to account for the significant class imbalance in the dataset (14.3% hits, 85.7% non-hits). AUPRC is used as the primary metric because it is more informative than accuracy or AUC in imbalance settings.

Our simplest baseline is a logistic regression model, which serves as a linear classifier over the input feature space. This acts as an important linear baseline for the project.

$$P(y = 1|x) = \sigma(w^T x + b)$$

Where w is the learned weight vector and b is the bias term. The model is trained by minimizing binary cross-entropy loss. The architecture can be summarized as:

Input(d=38) → FC(38 → 1) → Sigmoid → Output

We evaluated three gradient boosted tree models: XGBoost, LightGBM, and CatBoost. Gradient boosting builds an ensemble of weak learners sequentially, where each new tree corrects the residual errors of the previous ensemble. The prediction at boosting step m is:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where h_m is the newly added decision tree and η is the learning rate. All three models were configured with a learning rate of 0.05 to help with gradual, stable learning and reduce risk of overfitting. The specific hyperparameters for each model are as follows:

1. XGBoost: 500 trees, maximum depth of 5, subsample of 0.8
2. LightGBM: 1,000 trees, 63 leaves, feature and bagging fraction of 0.8
3. CatBoost: 500 iterations, maximum depth of 7, L2 regularization coefficient of 3.

These models are particularly well-suited to our tabular feature space since they capture non-linear feature interactions and provide feature importances. The XGBoost feature importance analysis revealed that track duration, genre embeddings, explicit content flags were among the most informative features.

Our advanced deep learning model is a **Wide & Deep Neural Network** that jointly learns from 38 engineered numeric features and a set of trainable genre embeddings. The core insight motivating this architecture is that music genre is not a simple categorical label; it is a distributed, overlapping signal that benefits from a learned representation rather than a one-hot encoding. At the same time, tabular audio features such as danceability, energy, and tempo exhibit both linear (wide) and non-linear (deep) relationships with hit prediction.

Each song is represented by two input streams:

- **Numeric stream (38 features):** 14 base Spotify audio features, 16 frozen mean-pooled genre embedding dimensions, and 8 hand-crafted interaction features. This stream is standardised using StandardScaler fit exclusively on the training set.
- **Genre ID stream (5 integers):** Each song's genre list is tokenised using a vocabulary of 1,045 unique genres. Up to 5 genres are encoded as integer indices; shorter lists are zero-padded. These IDs are passed into a trainable nn.Embedding layer within the network.

The model is a Wide & Deep architecture consisting of three components:

Genre Embedding Layer: An nn.Embedding(1045, 16, padding_idx=0) maps each song's genre token sequence to a 16-dimensional space. The five per-song genre vectors are mean-pooled to produce a single 16-dimensional genre representation. This layer is trained end-to-end, learning which genres are associated with chart success. After concatenation with the 38 numeric features, the combined input dimension is 54.

Wide Component: A single Linear(54 → 1) layer applied directly to the combined 54-dimensional input. This path captures global linear relationships, for example, the direct positive correlation between energy and hit probability, and acts as a memorisation mechanism.

Deep Component: A deeper non-linear path consisting of:

- Linear(54 → 256) + BatchNorm1d(256) + ReLU — input projection
- ResidualBlock 1: BatchNorm → ReLU → Dropout(0.35) → Linear(256→256) → BatchNorm → ReLU → Dropout(0.35) → Linear(256→256), with a skip connection
- ResidualBlock 2: same structure with Dropout(0.25) in both positions
- Head: Linear(256 → 64) → ReLU → Linear(64 → 1)

The final output logit is the sum of the wide and deep paths before applying sigmoid activation. Residual (skip) connections are critical for this architecture: they allow gradients to flow directly to earlier layers, preventing vanishing gradients and enabling the deep path to learn residual corrections on top of the wide path's linear estimate. The total parameter count is 313,096, all trainable.

Mathematically, the forward pass is:

$$z = W_{wide} \bullet x_{combined} + f_{deep}(x_{combined})$$

$$\hat{y} = \sigma(z)$$

where x_combined = [x_numeric ‖ mean_pool(Embed(genre_ids))], W_wide is the wide linear weight matrix, f_deep is the deep residual network, and σ is the sigmoid function applied only at inference (not during training, where BCEWithLogitsLoss handles the sigmoid internally for numerical stability).

In addition to the Wide & Deep NN, we trained five additional models as part of a broader ensemble pipeline:

LightGBM uses leaf-wise (best-first) tree growth with 63 leaves per tree, trained for up to 2,000 rounds with early stopping at 50 rounds on validation AUPRC. Key parameters: learning_rate=0.03, feature_fraction=0.8, bagging_fraction=0.8, lambda_l1=0.1, lambda_l2=1.0, is_unbalance=True.

CatBoost uses ordered boosting with symmetric trees, trained for up to 2,000 iterations with early stopping at 50. Key parameters: learning_rate=0.03, depth=7, l2_leaf_reg=3.0, auto_class_weights=Balanced, eval_metric=PRAUC.

XGBoost uses level-wise tree growth, trained for 500 estimators with early stopping at 40 rounds. Key parameters: learning_rate=0.05, max_depth=5, subsample=0.8, colsample_bytree=0.8, scale_pos_weight=5.99.

TabNet uses sequential attention-based feature selection. The best configuration from 5-fold CV was Config 5 (n_d=64, n_a=64, n_steps=5, gamma=1.5, lambda_sparse=0.001, lr=0.001) with a mean CV AUPRC of 0.2432.

Logistic Regression serves as a linear baseline with class_weight=balanced and L2 regularisation (C=0.1).

A two-level stacking ensemble was trained using out-of-fold (OOF) predictions. LightGBM, XGBoost, CatBoost, and Logistic Regression each generated predictions on 5 stratified folds of the training set. The resulting 4-column OOF prediction matrix was used to train a Logistic Regression meta-learner (C=0.5). This construction ensures the meta-learner never sees predictions made by a model trained on the same data, preventing it from learning to exploit overfitting artefacts. The meta-learner learned weights of +0.80 (LightGBM), +1.84 (XGBoost), +1.40 (CatBoost), and +0.97 (Logistic Regression), indicating that XGBoost and CatBoost carried the most signal in the stack.

The final model is a weighted average ensemble combining LightGBM (weight=0.50), XGBoost (weight=0.25), and the Wide & Deep NN (weight=0.25). These weights were selected by a grid search over integer weights [0, 1, 2, 3] for each of the 7 candidate models, normalised to sum to 1, and evaluated on the validation set AUPRC. Notably, the search assigned zero weight to TabNet, CatBoost, Logistic Regression, and the Stacking Ensemble, concentrating weight on the three models with the highest and most complementary individual AUPRC scores.

The Wide & Deep NN is trained using AdamW (Decoupled Weight Decay Regularisation, Loshchilov & Hutter 2019) with learning rate lr=1e-3 and weight_decay=1e-4. AdamW was chosen over standard Adam because it decouples the weight decay from the gradient update, which provides more reliable regularisation, particularly important for a model with 313,096 parameters trained on roughly 16,929 samples. Standard Adam absorbs weight decay into the adaptive learning rate scaling, which can lead to underregularisation in the final layers. AdamW avoids this by applying weight decay directly to the parameters.

We use **Binary Cross-Entropy with Logits Loss** (BCEWithLogitsLoss) with a positive class weight:

$$pos\_weight = \frac{|negative\ examples|}{|positive\ examples|} = \frac{14508}{2421} \approx 5.99$$

This directly reweights the loss for the minority (hit) class, making each hit example approximately 6× more costly than a non-hit. Without this, the model would converge to near-trivially predicting non-hits, achieving ~85% accuracy by ignoring the signal entirely. BCEWithLogitsLoss was preferred over a standalone sigmoid followed by BCELoss because the fused implementation is numerically more stable for extreme logit values.

For the gradient-boosted tree models, class imbalance is addressed through analogous reweighting mechanisms native to each library. XGBoost receives a **scale_pos_weight=5.99** argument, which multiplies the gradient and hessian contributions of positive examples by that factor during tree construction. LightGBM uses **is_unbalance=True**, which internally computes and applies a similar reweighting automatically. CatBoost uses **auto_class_weights=Balanced**, which scales class weights inversely proportional to their frequency. In all three cases the effect is equivalent: minority class (hit) examples contribute approximately 6× more to each boosting step than majority class (non-hit) examples, counteracting the model's natural tendency to optimise for the dominant class.

A ReduceLROnPlateau scheduler was applied to the W&D NN optimizer with mode=max (monitoring validation AUPRC), patience=5 epochs, and reduction factor=0.5. When the validation AUPRC fails to improve for 5 consecutive epochs, the learning rate is halved. This adaptive schedule reduces the learning rate from 1e-3 to 5e-4 as training progresses, allowing finer parameter updates in the later stages of optimisation without manual tuning of a fixed decay schedule.

Early stopping was applied to the W&D NN with patience=12 epochs, monitoring validation AUPRC rather than validation loss. This is a deliberate choice: optimising stopping on loss would halt training when the loss plateaus, which does not necessarily coincide with peak ranking performance on the imbalanced test set. Stopping on AUPRC directly aligns the training termination criterion with our primary evaluation metric. The model trained for 25 epochs before early stopping was triggered, with the best checkpoint at epoch 13 (val AUPRC=0.2910).

Gradient clipping with max_norm=1.0 was applied at each training step. Residual connections allow gradients to accumulate across skip paths, which can produce occasional gradient spikes that destabilise training. Clipping prevents these spikes from causing large parameter updates without affecting the overall learning dynamics during normal training steps.

All models output predicted probabilities. The decision threshold is tuned on the validation set by maximising F1 score across all candidate thresholds from the precision-recall curve, then fixed and applied to the test set. This ensures no information from the test set is used in threshold selection. The final model (Weighted Average Ensemble) used a threshold of 0.535.

Experimental Setup

Hardware & Software

Component	Specification

GPU	NVIDIA A100-SXM4-80GB
GPU VRAM	85.1 GB
Accelerator	CUDA
Framework	PyTorch
Python	3.12 (Google Colab default)
Key Libraries	scikit-learn, XGBoost, LightGBM, CatBoost, pytorch-tabnet
Environment	Google Colab

Compute Time

Wide & Deep Neural Network

Metric	Value
Average wall-clock time per epoch	**0.29 seconds**
Total epochs run (until early stopping)	**25**
Total training time	**7.2 seconds**
Batch size	**512**

Training samples	**16,929**

XGBoost

Metric	Value
Time per boosting round	**0.033 seconds**
Total rounds (best iteration)	**270**
Estimated total training time	**8.8 seconds**

Other Models

Model	Iterations / Epochs	Notes
LightGBM	194 rounds	Early stopping at 50, metric=average_precision
CatBoost	414 iterations	Early stopping at 50, metric=PRAUC
TabNet	Up to 109 epochs per fold	5-fold CV across 5 configs = 25 training runs
Stacking (OOF)	5 folds × 4 models	Each base model trained 5 times

Weighted Ensemble	< 1 second	Grid search over 4^7 weight combinations on val set

The full pipeline including all debugging runs, TabNet cross-validation, and stacking OOF training is estimated at approximately 1–2 GPU hours total across all development runs. The final clean run of the complete notebook took approximately 20–25 minutes end-to-end on the A100.

Hyperparameters

Wide & Deep Neural Network

Hyperparameter	Value	Justification
Learning rate	1e-3	Standard AdamW starting point for tabular DL
Weight decay	1e-4	Mild L2 regularisation via AdamW decoupling
Dropout (res1)	0.35	Higher dropout in earlier residual block
Dropout (res2)	0.25	Lower dropout in later block to preserve signal
Batch size	512	Balances gradient noise and throughput on A100
Genre embed dim	16	Sufficient capacity for 1,045 genres
Hidden dim	256	Empirically tuned; 128 underfit, 512 negligible gain

Residual blocks	2	Ablation showed 2 blocks optimal
Early stopping patience	12 epochs	On val AUPRC; allows LR schedule to kick in first
LR scheduler patience	5 epochs	Halves LR before early stop triggers
Gradient clip max_norm	1.0	Standard for residual networks
Pos weight	5.99	Computed as neg/pos = 14508/2421
Max epochs	100	Upper bound; early stopping triggers first

Gradient-Boosted Trees

Parameter	LightGBM	XGBoost	CatBoost
n_estimators / iterations	2000 (best: 194)	500 (best: 270)	2000 (best: 414)
Learning rate	0.03	0.05	0.03
Max depth / leaves	63 leaves	depth=5	depth=7
Row subsampling	0.8 (bagging_fraction)	0.8 (subsample)	—

Column subsampling	0.8 (feature_fraction)	0.8 (colsample_bytree)	—
Regularisation	l1=0.1, l2=1.0	—	l2_leaf_reg=3.0
Imbalance handling	is_unbalance=True	scale_pos_weight=5.99	auto_class_weights=Balanced
Primary train metric	average_precision	aucpr	PRAUC
Early stopping rounds	50	40	50
Random seed	426	426	426

TabNet (Best Config — Config 5)

Parameter	Value
n_d / n_a	64 / 64
n_steps	5
gamma	1.5
lambda_sparse	0.001
Learning rate	0.001

Weight decay	1e-5
Optimizer	AdamW
Mask type	entmax
Batch size	1024
Virtual batch size	128
CV folds	5 (StratifiedKFold)
CV AUPRC (best config)	0.2432 ± 0.0110

Stacking Ensemble

Parameter	**Value**
Base models	LightGBM, XGBoost, CatBoost, Logistic Regression
CV strategy	5-fold StratifiedKFold on training set
Meta-learner	Logistic Regression (C=0.5)
Meta-learner weights (LGB/XGB/CAT/LR)	+0.80 / +1.84 / +1.40 / +0.97

Weighted Average Ensemble (Final Model)

Model	Weight
LightGBM	0.50
XGBoost	0.25
Wide & Deep NN	0.25
TabNet	0.00
CatBoost	0.00
Logistic Regression	0.00
Stacking Ensemble	0.00

Weights were selected by grid search over [0, 1, 2, 3] per model (normalised to sum to 1), evaluated on the hold-out validation set AUPRC. The test set was never accessed during weight selection.

Results

Model performance was evaluated using AUPRC as the primary metric, with ROC-AUC, F1 score, precision, recall, and accuracy reported as secondary measures. Because only 14.3% of songs in the dataset were labeled as hits, AUPRC provides the most meaningful measure of minority-class ranking performance, while raw accuracy alone can be misleading under severe class imbalance.

The baseline Logistic Regression established that audio features and engineered feature interactions contain measurable predictive signal. Logistic regression achieved an AUPRC of 0.2398 and ROC-AUC of 0.6739, substantially outperforming the random baseline AUPRC of approximately 0.143. However, its precision remained low at 0.2124 despite strong recall of 0.6724, indicating that the model tended to over-predict hit songs in order to recover minority cases. This produced an F1 score of 0.3228 and accuracy of only 0.5965, confirming that linear decision boundaries were insufficient for fully separating hit and non-hit classes.

Tree-based boosting methods produced the strongest single-model results. XGBoost improved AUPRC to 0.3166 and ROC-AUC to 0.7197, with an F1 score of 0.3735. Its recall of 0.5877 remained relatively strong while precision improved to 0.2738, demonstrating a more balanced minority-class tradeoff than logistic regression. LightGBM achieved the strongest single-model AUPRC at 0.3278 and ROC-AUC of 0.7212, while also producing the highest accuracy among individual models at 0.7533. LightGBM's stronger precision (0.2906) came with lower recall (0.5029), reflecting a more conservative classification strategy. CatBoost performed nearly identically, with AUPRC of 0.3237 and ROC-AUC of 0.7201, suggesting that all three gradient boosting methods converged toward a similar performance ceiling.

The Wide & Deep neural network with genre embeddings offered competitive but not superior performance relative to tree-based methods. This architecture reached an AUPRC of 0.3083 and ROC-AUC of 0.7099, outperforming logistic regression but falling short of the best boosted trees. Its recall of 0.6455 was notably high, second only to logistic regression, but precision remained relatively low at 0.2476. This indicates that the neural network was effective at recovering hit songs but less selective when labeling positives. Despite approximately 313,000 parameters and residual connections, gains over boosting models remained modest.

TabNet underperformed relative to all major alternatives, with AUPRC of 0.2147 and ROC-AUC of 0.6267. Although TabNet is designed for tabular learning, its performance suggests that the feature space may not have contained the kind of sparse structured interactions where TabNet typically excels. Early stopping occurred quickly, further indicating limited generalization benefit from this architecture.

Ensemble methods provided the strongest overall results, though gains remained incremental rather than transformative. The stacking ensemble achieved an AUPRC of 0.3146 and ROC-AUC of 0.7202, slightly below the best tree models despite combining four base learners through a logistic regression meta-model. In contrast, the weighted average ensemble delivered the strongest overall performance with an AUPRC of 0.3319, ROC-AUC of 0.7246, and F1 score of 0.3782. This represents a 2.32× improvement over the random baseline in AUPRC. However, the improvement over LightGBM alone was only 0.0041 AUPRC, indicating that most available predictive signal had already been captured by the strongest individual tree learner.

The optimal weighted ensemble assigned nearly all predictive weight to LightGBM and the Wide & Deep neural network, with smaller contribution from TabNet and zero contribution from logistic regression, XGBoost, CatBoost, and stacking. This suggests that LightGBM and the neural network captured complementary structure, while several other models produced largely redundant predictions.

Overall, the results show that increasingly sophisticated architectures improved ranking quality, but gains became progressively smaller as complexity increased. This pattern suggests that limitations arise less from modeling capacity and more from the inherent informational ceiling of audio-derived features.

Discussion, Ablations, and Error Analysis

The strongest conclusion from the modeling results is that architecture complexity produced diminishing returns. While moving from logistic regression to boosted trees generated substantial gains, additional complexity beyond gradient boosting yielded only marginal improvement. The weighted ensemble achieved the best overall AUPRC of 0.3319, but this exceeded LightGBM by only 0.0041, a very small practical margin given the additional modeling complexity required.

This pattern suggests that most learnable signal in the dataset is already captured by nonlinear feature interactions within gradient-boosted trees. Logistic regression improved substantially over random guessing, demonstrating that audio features contain real predictive information, but its lower AUPRC and low precision indicate that linear boundaries cannot adequately model hit-song structure. Once nonlinear boosting methods were introduced, performance increased sharply, confirming that interactions among tempo, loudness, energy, danceability, and engineered combinations are important for separating hits from non-hits.

However, the near-convergence of XGBoost, LightGBM, and CatBoost strongly suggests a feature ceiling. All three boosting algorithms produced AUPRC values between 0.3166 and 0.3278, despite different optimization strategies. This consistency implies that model family matters less than feature informativeness once nonlinear capacity is sufficient.

The Wide & Deep neural network offers important ablation insight. Although the network introduced genre embeddings, residual blocks, and over 300,000 trainable parameters, its AUPRC remained below LightGBM. This suggests that learned dense representations did not uncover substantially new signal beyond what engineered tabular features already encoded. However, its relatively strong recall indicates that genre embeddings likely contributed meaningful contextual information that tree models did not fully exploit.

The weighted ensemble confirms this interpretation. The final optimal weighting assigned equal dominant weight to LightGBM and the neural network, while assigning zero weight to logistic regression, XGBoost, CatBoost, and stacking outputs. This means that LightGBM and the neural network contributed complementary predictive structure, whereas the remaining models added little independent information.

Error behavior further supports the interpretation that external variables drive much of hit formation. Models frequently produced false positives for songs with highly conventional commercial audio profiles: strong energy, mainstream tempo, and high danceability, yet no commercial success. Conversely, false negatives often occurred for songs that became hits despite atypical acoustic characteristics.

These errors are not random—they reveal that commercial success depends heavily on variables absent from the feature set. Audio characteristics explain part of hit probability, but factors such as marketing

investment, artist reputation, release timing, platform placement, viral exposure, and cultural momentum likely explain much of the residual variance.

The relatively high recall observed in several models also reflects an intentional tradeoff created by threshold tuning. Because minority hit detection is difficult, thresholds were selected to improve positive recovery at the cost of precision. This explains why some models maintain recall above 0.60 while precision remains below 0.30.

Overall, the results align strongly with the project's central hypothesis: there is measurable structure in hit songs, but no purely audio-based formula fully determines success. Audio features alone can more than double random baseline performance, yet even advanced ensembles leave substantial uncertainty unresolved.

Future improvements should focus less on new architectures and more on richer feature sources. Temporal artist history, prior chart momentum, social media activity, release timing, and broader contextual metadata are likely necessary to move performance meaningfully beyond the current ceiling.

Conclusions

This study investigated whether intrinsic musical features alone can meaningfully predict song success. Overall, the problem can be considered partially solved. The developed models demonstrated clear predictive signals relative to naive baselines, indicating that audio-derived characteristics contain non-trivial information about a song's likelihood of becoming a hit. However, predictive performance plateaued across successive architectural refinements, and classification accuracy remained constrained by persistent false positives, suggesting that intrinsic musical attributes alone are insufficient to fully capture dynamics of chart success.

Several limitations contributed to these outcomes. First, the usable dataset was substantially reduced to avoid extensive imputation, limiting both sample size and feature diversity. Second, class imbalance posed a significant challenge, as hit songs represent only a small fraction of the overall population. Third, the genre variable introduced high variability, with some tracks assigned numerous overlapping genre labels (e.g. "pop rock", "rock", "pop"). While embedding techniques partially mitigated this issue, a more structured or hierarchical genre representation would likely improve predictive power. Finally, the absence of contextual industry variables, such as marketing investment or artist popularity, constrained the explanatory scope of the models.

Assuming a six-month extension and increased computational resources, future work would build directly on one of the central findings of this study: that intrinsic musical features alone exhibit limited predictive power for determining whether a song is a hit. Thus, methodological extensions would primarily focus on incorporating extrinsic contextual variables, such as promotional investment, artist historical performance, release timing, and platform exposure, to construct a more holistic predictive framework. We would also transition from binary classification toward continuous hit-score modeling, enabling more nuanced representations of success that account for peak chart position, longevity, and trajectory of growth.

Furthermore, given the highly imbalanced nature of hit prediction, future work would explore the use of cost-sensitive learning to better align the training objective with the rarity of successful songs.

In summary, while continued efforts to refine intrinsic musical features remain valuable, the findings of this study suggest that meaningful improvements in predictive performance will primarily depend on incorporating broader contextual and industry-related variables. Future work should therefore prioritize the integration of extrinsic predictors, the transition to continuous hit-score modeling, and the development of more robust approaches for addressing class imbalance during training. In addition, systematic ablation analyses will be essential for clarifying the relative contributions of musical attributes and external factors, thereby enhancing both the interpretability and generalizability of predictive models. Collectively, these directions will support the development of more comprehensive and realistic frameworks for understanding the complex determinants of commercial music success.

References & Appendix

Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive interpretable tabular learning. Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), 6679–6687. https://doi.org/10.1609/aaai.v35i8.16826

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Deruty, E., Grachten, M., Lattner, S., Nistal, J., & Aouameur, C. (2022). On the development and practice of AI technology for contemporary popular music production. Transactions of the International Society for Music Information Retrieval, 5(1), 35–49. https://doi.org/10.5334/tismir.100

Dhanaraj, R., & Logan, B. (2005). Automatic prediction of hit songs. In Proceedings of the 6th International Society for Music Information Retrieval Conference (pp. 488–491).

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems (Vol. 30).

Pachet, F., & Roy, P. (2008). Hit song science is not yet a science. In Proceedings of the 9th International Society for Music Information Retrieval Conference (pp. 355–360).

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In Advances in Neural Information Processing Systems (Vol. 31).

Roads, C. (1985). Research in music and artificial intelligence. ACM Computing Surveys, 17(2), 163–190. https://doi.org/10.1145/4468.4469

Rompolas, G., Smpoukis, A., Kafeza, E., & Makris, C. (2024). Predicting song popularity through machine learning and sentiment analysis on social networks. IFIP Advances in Information and Communication Technology, 314–324. https://doi.org/10.1007/978-3-031-63227-3_22

Vavaroutsos, P., & Vikatos, P. (2024). HSP-TL: A deep metric learning model with triplet loss for hit song prediction using lyrics and audio features. Science Talks, 10, Article 100363. https://doi.org/10.1016/j.sctalk.2024.100363

Yang, L.-C., Chou, S.-Y., Liu, J.-Y., Yang, Y.-H., & Chen, Y.-A. (2017). Revisiting the problem of audio-based hit song prediction using convolutional neural networks. In 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 621–625). https://doi.org/10.1109/ICASSP.2017.7952230

Contributions Statement

Steven Li:

- Led data-pre-processing and TabNet implementation and took initiative alongside Ethan in coordinating team direction; contributed to presentation slides and Final Report Sections 2, 4, and 5

Daniel Yao:

- Conducted EDA and baseline modeling, while providing flexible support across team teams; contributed to Final Report Sections 1, 3, and 5

Tim Wu:

- Focused on hands-on technical work including ensembling, final model experimentation, and ablation studies; contributed to Final Report Sections 5 and 6

Ethan Biddle:

- Designed Wide & Deep Neural Network while taking primary role in organizing, planning, and keeping the team aligned; contributed to presentation slides and Final Report Sections 7 and 8

Codebase

Github: https://github.com/listeven06/msd-hit-prediction/tree/main